

Mockito Hands-On Exercises

Exercise 1: Mocking and Stubbing

Scenario:

You need to test a service that depends on an external API. Use Mockito to mock the external API and stub its methods.

Steps:

1. Create a mock object for the external API.
2. Stub the methods to return predefined values.
3. Write a test case that uses the mock object.

Solution:

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <scope>test</scope>
  </dependency>
  <!-- Optionally: parameterized tests support -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-params</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>5.12.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

DataProvider.java

```
package com.example;
```

```
public interface DataProvider {
```

```
    String loadData();
```

```
}
```

DataManager.java

```
package com.example;

public class DataManager {

    private DataProvider provider;

    public DataManager(DataProvider provider) {
        this.provider = provider;
    }

    public String displayData() {

        return provider.loadData();

    }
}
```

DataManagerTest.java

```
package com.example;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.when;

import org.junit.jupiter.api.Test;

public class DataManagerTest {

    @Test
    public void testDisplayData() {
```

```

DataProvider provider = mock(DataProvider.class);

when(provider.loadData()).thenReturn("Mockito Success");

DataManager manager = new DataManager(provider);

String output = manager.displayData();

System.out.println("Output : " + output);

assertEquals("Mockito Success", output);
}
}

```

Output:

The screenshot displays the Spring Tools for Eclipse IDE interface. The central editor shows the `DataManagerTest.java` file with the following code:

```

1 package com.example;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import static org.mockito.Mockito.mock;
5 import static org.mockito.Mockito.when;
6
7 import org.junit.jupiter.api.Test;
8
9 public class DataManagerTest {
10
11     @Test
12     public void testDisplayData() {
13
14         DataProvider provider = mock(DataProvider.class);
15
16         when(provider.loadData()).thenReturn("Mockito Success");
17
18         DataManager manager = new DataManager(provider);
19
20         String output = manager.displayData();
21
22         System.out.println("Output : " + output);
23
24         assertEquals("Mockito Success", output);
25     }
26 }

```

The left sidebar shows the Package Explorer with the project structure and the JUnit console indicating the test was successful. The bottom terminal window shows the following output:

```

<terminated> DataManagerTest [JUnit] C:\Program Files\Java\jdk-21\bin\javaw.exe (1 Jul 2026, 12:21:58 pm - 12:22:01 pm elapsed 0:00:02) [pid: 32948]
WARNING: A Java agent has been loaded dynamically (C:\Users\upend\m2\repository\net\bytebuddy\byte-buddy-agent\1.14.15\byte-buddy-agent-1.14.15.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disabled by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been modified
Output : Mockito Success

```

Exercise 2: Verifying Interactions

Scenario:

You need to ensure that a method is called with specific arguments.

Steps:

1. Create a mock object.
2. Call the method with specific arguments.
3. Verify the interaction.

Solution:

NotificationService:

```
package com.example;
```

```
public interface NotificationService {
```

```
    void sendMessage(String mobileNumber);
```

```
}
```

CustomerService.java:

```
package com.example;
```

```
public class CustomerService {
```

```
    private NotificationService notification;
```

```
    public CustomerService(NotificationService notification) {
```

```
        this.notification = notification;
```

```
    }
```

```
    public void addCustomer(String mobileNumber) {
```

```
        notification.sendMessage(mobileNumber);
```

```
    }
```

```
}
```

```
CustomerServiceTest.java:package com.example;
```

```
import static org.mockito.Mockito.mock;
```

```
import static org.mockito.Mockito.verify;
```

```
import org.junit.jupiter.api.Test;
```

```
public class CustomerServiceTest {
```

```
    @Test
```

```
    public void testAddCustomer() {
```

```
        NotificationService notify = mock(NotificationService.class);
```

```
        CustomerService customer = new CustomerService(notify);
```

```
        String number = "9876543210";
```

```
        customer.addCustomer(number);
```

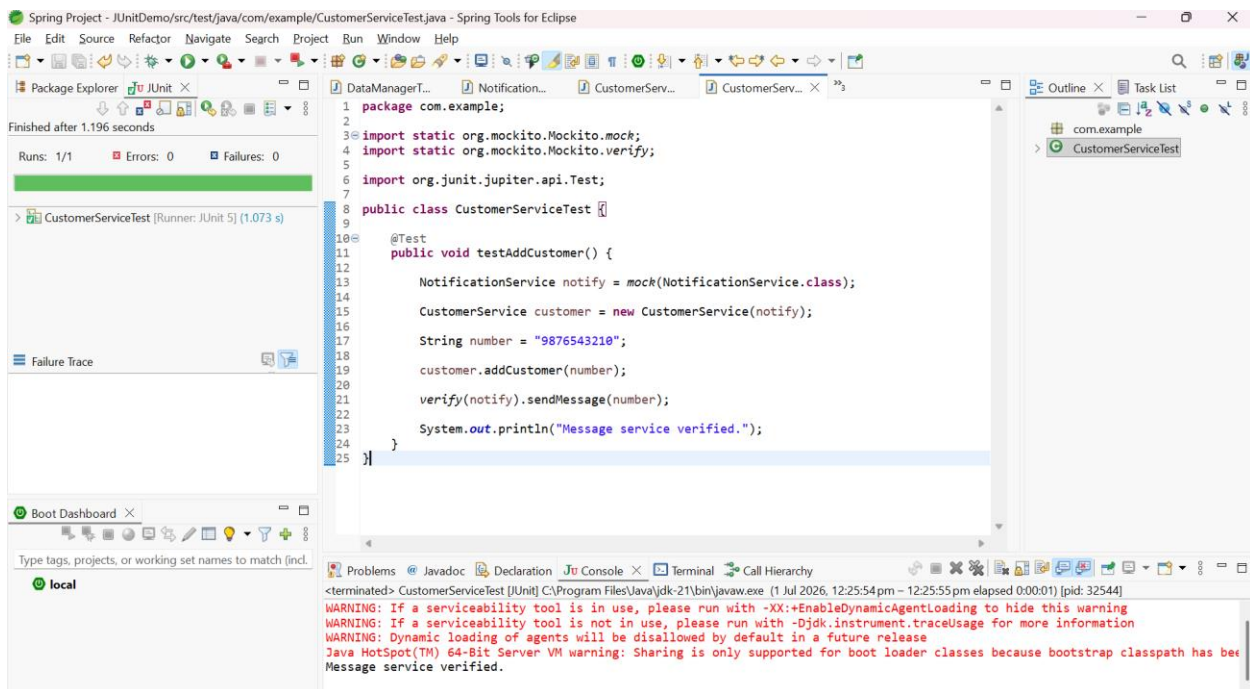
```
verify(notify).sendMessage(number);
```

```
System.out.println("Message service verified.");
```

```
}
```

```
}
```

Output:



The screenshot displays the Eclipse IDE interface. The main editor shows the `CustomerServiceTest.java` file with the following code:

```
1 package com.example;
2
3 import static org.mockito.Mockito.mock;
4 import static org.mockito.Mockito.verify;
5
6 import org.junit.jupiter.api.Test;
7
8 public class CustomerServiceTest {
9
10     @Test
11     public void testAddCustomer() {
12
13         NotificationService notify = mock(NotificationService.class);
14
15         CustomerService customer = new CustomerService(notify);
16
17         String number = "9876543210";
18
19         customer.addCustomer(number);
20
21         verify(notify).sendMessage(number);
22
23         System.out.println("Message service verified.");
24     }
25 }
```

The left sidebar shows the Package Explorer with the project structure, and the JUnit console indicates the test was successful, running in 1.073 seconds. The bottom console shows the output of the test, including the printed message and several warnings related to the Java HotSpot VM.

```
<terminated> CustomerServiceTest [JUnit] C:\Program Files\Java\jdk-21\bin\javaw.exe (1 Jul 2026, 12:25:54 pm - 12:25:55 pm elapsed 0:00:01) [pid: 32544]
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been modified
Message service verified.
```